

TYPES OF MULTIPLE INHERITANCE

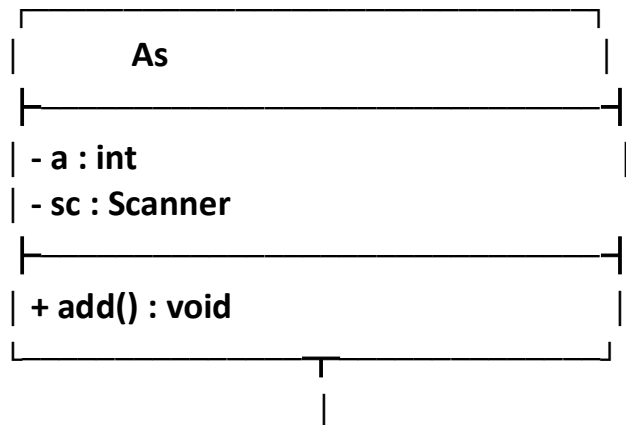
1. C1 Extends C2

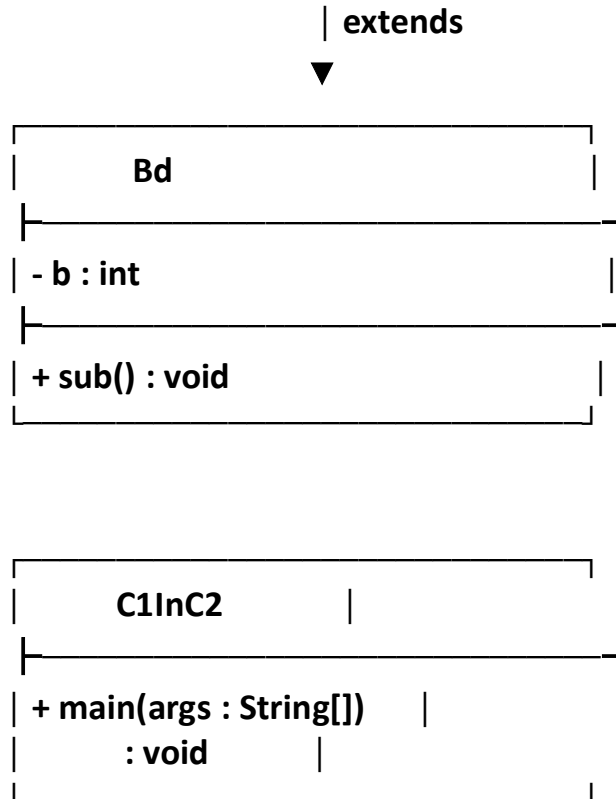
objective : write the code which accepts the class1 extends the class2

Algorithm :

1. Start.
2. Define a class As with an integer variable a and a Scanner object.
3. Define the add() method in class As.
4. Read the value of a from the user and display it.
5. Define a class Bd that extends class As.
6. Declare an integer variable b in class Bd.
7. Define the sub() method in class Bd.
8. Read the value of b from the user.
9. Calculate the difference between a and b using $a - b$.
10. Display the difference.
11. In the main() method, create an object bd of class Bd.
12. Call bd.add() to read and display the value of a.
13. Call bd.sub() to read b and display the difference.
14. Stop.

UML DIAGRAM :





SOURCE CODE :

```
package assignment2;

import java.util.Scanner;

class As{
    int a;
    Scanner sc=new Scanner(System.in);
    void add() {
        System.out.println("enter the value of a :");
        a=sc.nextInt();
        System.out.println("the value of x is :"+ a);
    }
}
class Bd extends As{
    int b;
    void sub() {
        System.out.println("enter the value of b :");
```

```

        b=sc.nextInt();
        System.out.println("the diff of a and b is :"+(a-b));
    }
}
public class C1InC2 {
    public static void main(String[] args) {
        Bd bd=new Bd();
        bd.add();
        bd.sub();
    }
}

```

OUTPUT :

```

enter the value of a :
23
the value of x is :23
enter the value of b :
32
the diff of a and b is :-9

```

2. C1 Extends C2,C3 is not possible in classes

3. C1 implements I1

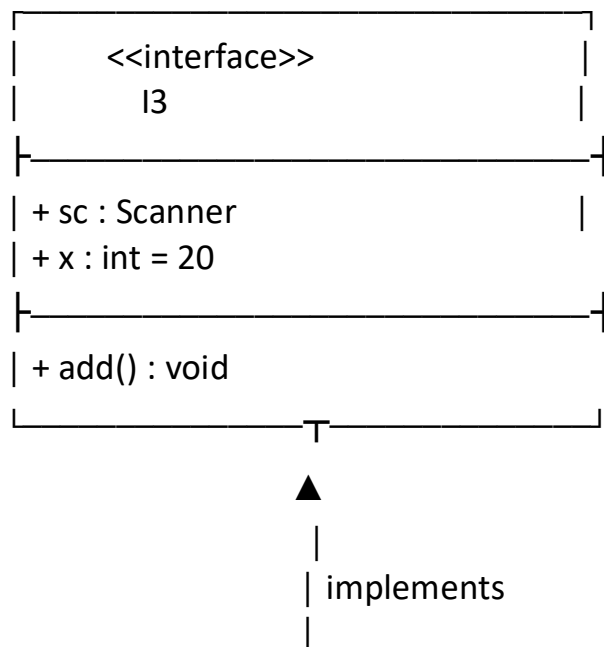
objective : to write the program of class 1 implements interface 1

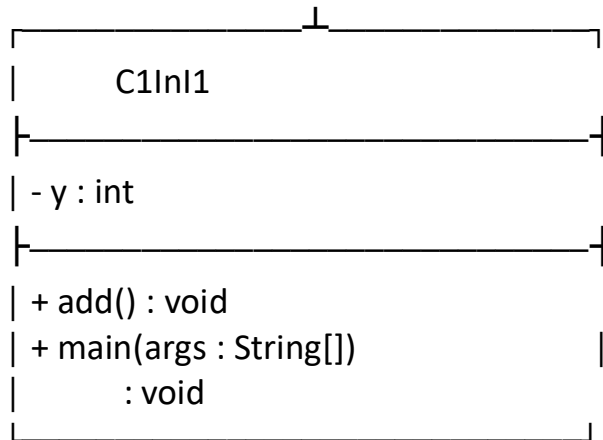
Algorithm :

1. Start.

2. Import the Scanner class.
3. Define an interface I3.
4. Declare a Scanner object sc in interface I3.
5. Declare and initialize an integer variable x with value 20.
6. Declare an abstract method add() in interface I3.
7. Define a class C1InI1 that implements interface I3.
8. Declare an integer variable y in class C1InI1.
9. Override the add() method in class C1InI1.
10. Read the value of y from the user.
11. Display the value of x.
12. Calculate the sum of x and y using $x + y$.
13. Display the sum of two numbers.
14. Perform integer division of x by y using x / y .
15. Display the result of division.
16. If y is zero, catch the ArithmeticException.
17. Display the message "the value of y is not zero".
18. In the main() method, create an object cs of class C1InI1.
19. Call cs.add() to execute the add() method.
20. Stop.

UML DIAGRAM :





SOURCE CODE :

```

package assignment2;

import java.util.Scanner;

interface I56{
    Scanner sc=new Scanner(System.in);
    //in interfaces, we must initialize the variables
    int x=20;
    // int e; wrong
    void add();
}

public class C1In1 implements I56{
    int y; //in classes, it's our wish to initialize variables
    @Override
    public void add() {
        System.out.println("enter the value of y");
        y=sc.nextInt();
        System.out.println("the value of x is"+x);
        System.out.println("the sum of two numbers is :"+(x+y));
        try {
            int r=x/y;
            System.out.println("the division of two numbers is:"+r);
        } catch (ArithmeticException ae){
            System.out.println("the value of y is not zero");
        }
    }
}
  
```

```

    public static void main(String[] args) {
        C1InI1 cs=new C1InI1();
        cs.add();
    }
}

```

OUTPUT :

```

enter the value of y
20
the value of x is20
the sum of two numbers is :40
the division of two numbers is:1

Process finished with exit code 0

```

4. C1 implements I1,I2

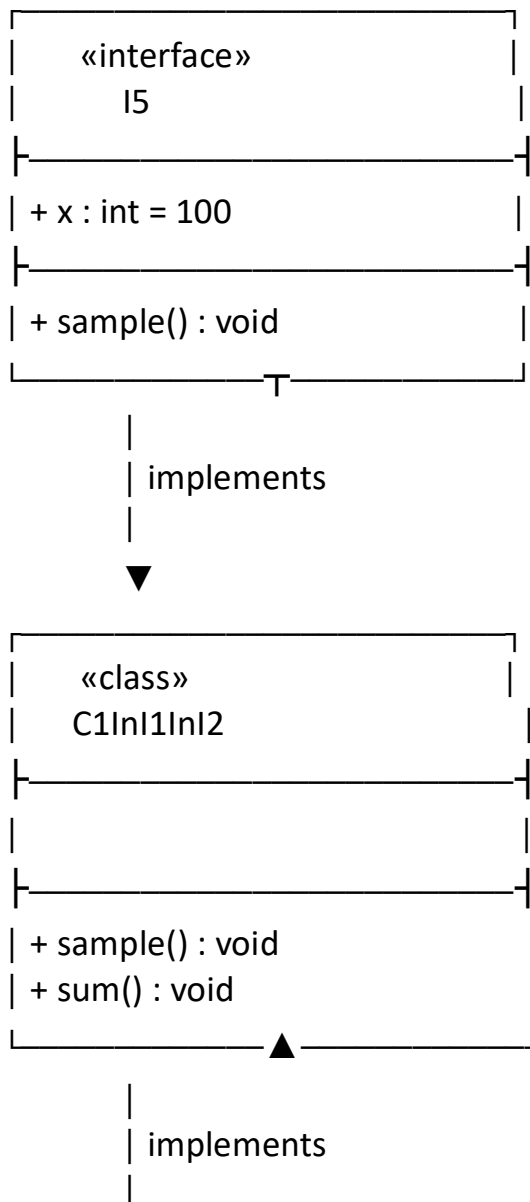
Objective :write the program of C1 implements I1,I2

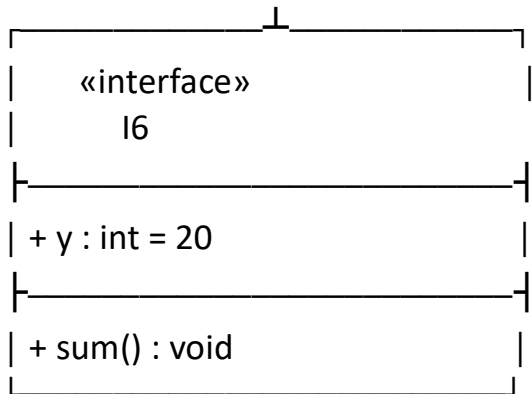
Algorithm :

1. Start.
2. Define an interface I1.
3. Declare and initialize an integer variable x with value 100 in interface I1.
4. Declare an abstract method sample() in interface I1.
5. Define an interface I2.
6. Declare and initialize an integer variable y with value 20 in interface I2.
7. Declare an abstract method sum() in interface I2.
8. Define a class Interface that implements interfaces I1 and I2.
9. Override the sample() method of interface I1.
10. Display the message "this is in I1".
11. Calculate the sum of x and y using $x + y$.

12. Display the sum of the numbers.
13. Override the sum() method of interface I2.
14. Display the message "this is in I2".
15. Calculate the difference between x and y using x - y.
16. Display the difference of two numbers.
17. In the main() method, create an object obj of class Interface.
18. Call obj.sample() to execute the sample() method.
19. Call obj.sum() to execute the sum() method.
20. Stop.

UML DIAGRAM :





SOURCE CODE :

```

interface I5{
    int x=100;
    abstract void sample();
}
interface I6{
    int y=20;
    abstract void sum();
}
  
```

```

public class C1InI1InI2 implements I1,I2 {
  
```

```

    @Override
    public void sample() {
        System.out.println("this is in I1");
        System.out.println("the sum of the numbers is" +(x+y));
    }
    @Override
    public void sum()
    {
        System.out.println("this is in I2");
        System.out.println("the sub of two numbers is :"+(x-y));
    }
    public static void main(String[] args) {
        Interface obj=new Interface();
        obj.sample();
        obj.sum();
    }
  
```

```

    }
} interface I5{
    int x=100;
    abstract void sample();
}
interface I6{
    int y=20;
    abstract void sum();
}

```

```

public class C1InI1InI2 implements I1,I2 {

```

```

    @Override
    public void sample() {
        System.out.println("this is in I1");
        System.out.println("the sum of the numbers is" +(x+y));
    }
    @Override
    public void sum()
    {
        System.out.println("this is in I2");
        System.out.println("the sub of two numbers is :"+(x-y));
    }
    public static void main(String[] args) {
        Interface obj=new Interface();
        obj.sample();
        obj.sum();
    }
}

```

OUTPUT:

```

this is in I1
the sum of the numbers is120
this is in I2
the sub of two numbers is :80

```

```

Process finished with exit code 0

```

5. C1 implements C2 Extends I1

Objective :

write the java program for the C1 implements C2 extends I1

**Error is there because the class must extends the class only ,
class not extends interfaces.class implements interface.**

interface extends interface only .

6.I1 extends I2

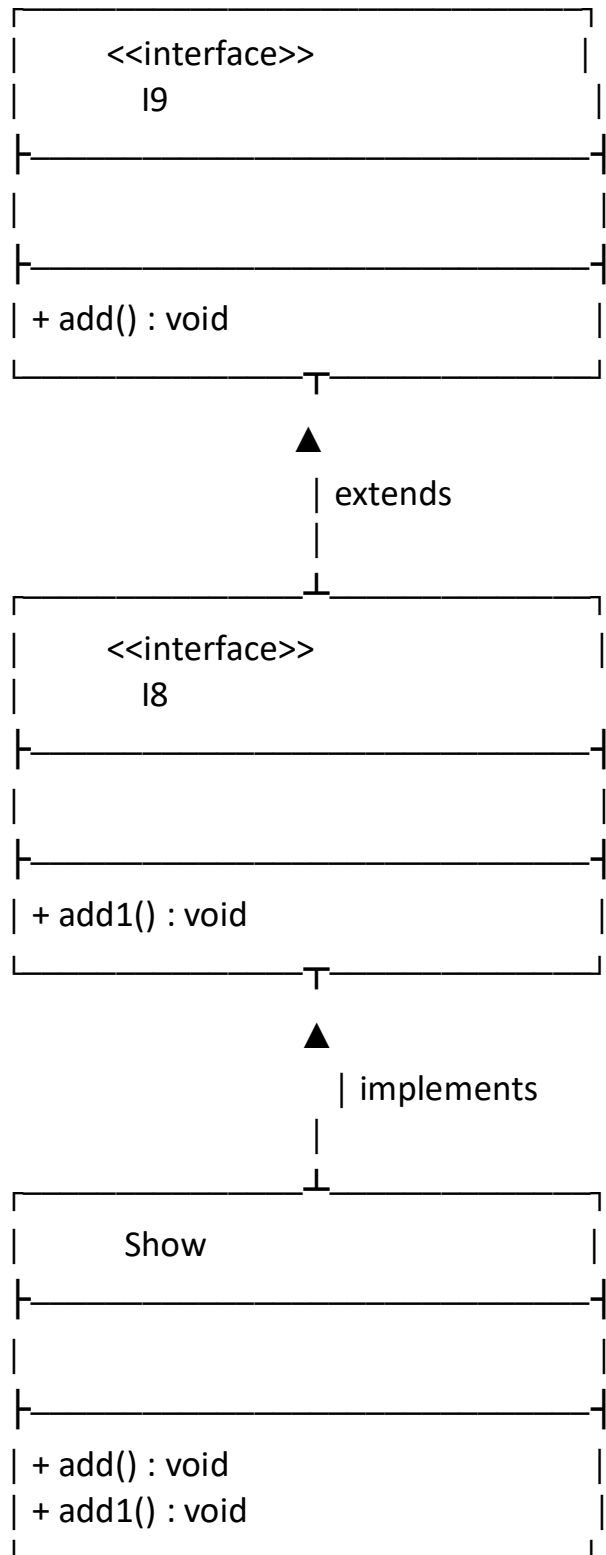
Objective :

write the program that interface extends the another interface

Algorithm :

1. Start.
2. Define an interface I9.
3. Declare an abstract method add() in interface I9.
4. Define an interface I8 that extends interface I9.
5. Declare an abstract method add1() in interface I8.
6. Define a class Show that implements interface I8.
7. Override the add() method inherited from interface I9.
8. Display "I9 method".
9. Override the add1() method declared in interface I8.
10. Display "I8 method".
11. In the main() method, create an object s of class Show.
12. Call s.add() to execute the add() method.
13. Call s.add1() to execute the add1() method.
14. Stop.

UML DIAGRAM:



SOURCE CODE:

```
interface I9{
    void add();
}
interface I8 extends I9{
    void add1();
}
class Show implements I8{
    @Override
    public void add() {
        System.out.println("I9 method");
    }

    @Override
    public void add1() {
        System.out.println("I8 method");
    }
}

public class I1ExtendsI2 {
    public static void main(String[] args) {
        Show s=new Show();
        s.add();
        s.add1();
    }
}
```

OUTPUT :

```
"C:\Program Files\Java\jdk-24\bin\j
I9 method
I8 method

Process finished with exit code 0
```

7.I1 implements C1

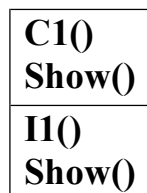
Objective :

write the program with interface implements class

Algorithm

1. Start the program.
2. Create a class C1.
3. Create an interface I1.
4. Try to make I1 implement C1.
5. Observe that Java does not allow an interface to implement a class.
6. Stop the program.

UML Class Diagram



Source Code

```
class C1 {  
    void show(){  
        System.out.println("C1 method");  
    }  
}  
  
// ERROR: An interface cannot implement a class  
interface I1 implements C1 {  
    void show1();  
}
```

Output / Error

Compilation Error

error: '{' expected

interface I1 implements C1

8. I1 extends I2,I3

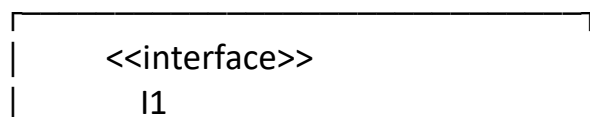
Objective :

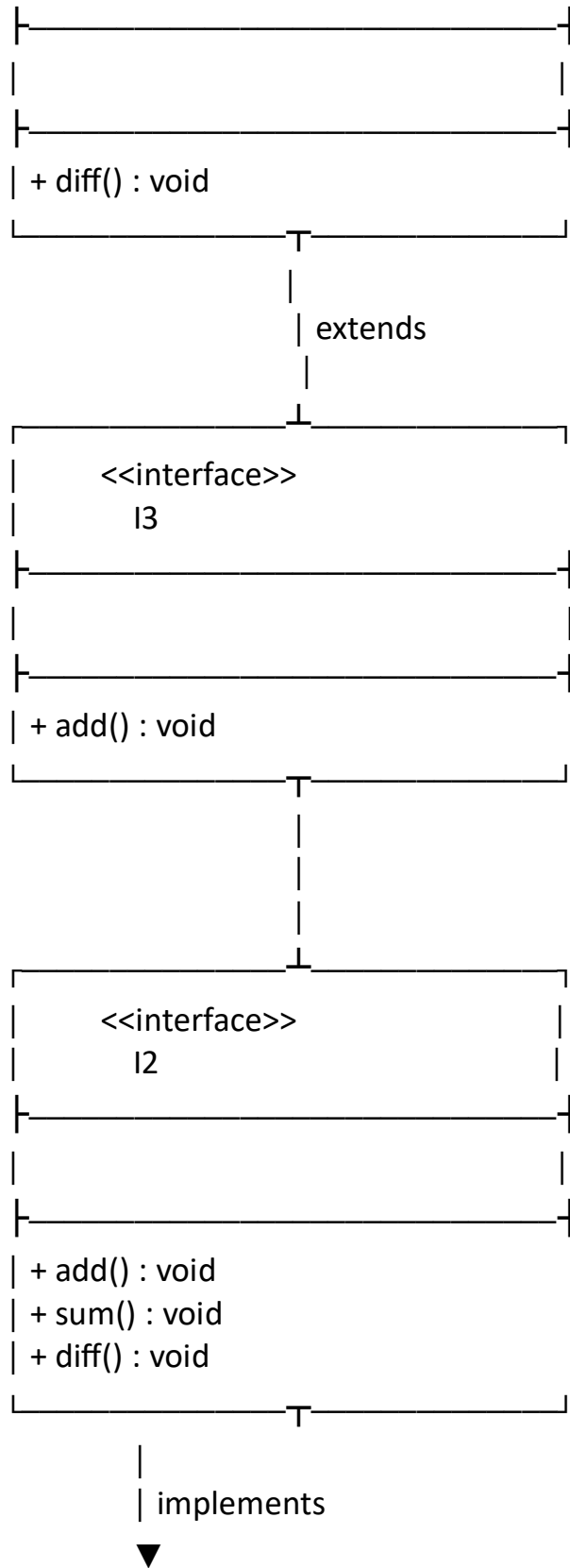
write the program with interface extends two another interfaces

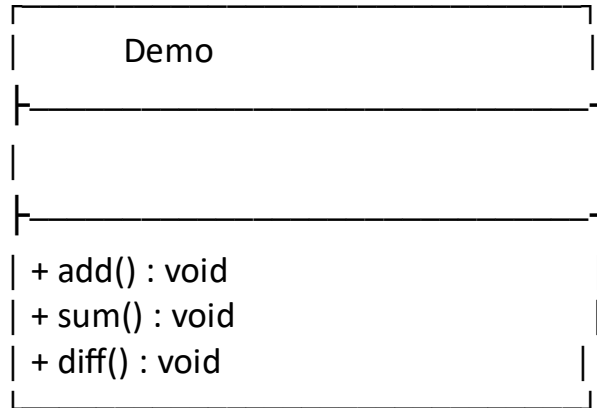
Algorithm :

1. Start.
2. Define an interface I1.
3. Declare an abstract method diff() in interface I1.
4. Define an interface I2.
5. Declare abstract methods add(), sum(), and diff() in interface I2.
6. Define an interface I3 that extends both I1 and I2.
7. Declare the add() method in interface I3.
8. Define a class Demo that implements interface I2.
9. Override the add() method in class Demo.
10. Display "I3 method".
11. Override the sum() method in class Demo.
12. Display "I2 method".
13. Override the diff() method in class Demo.
14. Display "I1 method".
15. In the main() method, create an object d of class Demo.
16. Call d.diff() to execute the diff() method.
17. Call d.sum() to execute the sum() method.
18. Call d.add() to execute the add() method.
19. Stop.

UML DIAGRAM :







SOURCE CODE :

```
package assignment2;
```

```
interface I1 {  
    void diff(); // in interface the methods don't have any body like in class  
}
```

```
interface I2 {  
    void add();  
  
    void sum();  
  
    void diff();  
}
```

```
interface I3 extends I1, I2 {  
    void add();  
}
```

```
class Demo implements I2 {  
  
    @Override  
    public void add(){  
        System.out.println("I3 method");  
    }  
  
    @Override  
    public void sum() {  
        System.out.println("I2 method");  
    }  
}
```

```

    }

    @Override
    public void diff() {
        System.out.println("I1 method");
    }
}

public class I1ExtendsI2I3 {

    public static void main(String[] args) {

        Demo d = new Demo();

        d.diff();
        d.sum();
        d.add();
    }
}

```

OUTPUT :

```

C:\Program Files\Java\jdk-9.0.4\bin> java I1ExtendsI2I3
I1 method
I2 method
I3 method

Process finished with exit code 0

```

